

Beyond Final Answers: Trajectory-Level Hallucination Detection in Multi-Agent Industrial Workflows

STATGR5293 — Generative AI Using LLMs — Final Project Report

Columbia University, Spring 2026 Section 007 (Wednesday)
Instructor: Parijat Dube Project PI: Dhaval C. Patel (IBM Research)

May 2026

Contents

Abstract	3
1. Introduction	4
2. Related Work	5
3. Problem Formulation	6
3.1 Trajectory Structure	6
3.2 Hallucination Taxonomy	7
3.3 Detection Tasks	7
3.4 Research Questions	8
4. Methodology	8
4.1 Evaluation Pipeline	8
4.2 Detection Modeling	8
4.3 LLM-as-a-Judge	9
4.4 Signal Analysis	9
5. Trajel Dataset	9
5.1 Composition	9
5.2 Annotation Protocol	10
5.3 Annotation Quality	10
5.4 Type Distribution	11
5.5 Cross-Model Analysis	12
5.6 Localization Analysis	12
6. Experiments and Results	12
6.1 Experimental Setup	12
6.2 Per-Type Detection Quality (RQ1)	13
6.3 Binary vs. Taxonomy-Aware Evaluation (RQ2)	13
6.4 Supervised Detection Models (RQ3)	13

6.5 Signal Analysis (RQ4)	14
7. Discussion	15
7.1 Why Procedural Hallucinations Dominate	15
7.2 The Multi-Label Problem	15
7.3 Execution Signals as Practical Guardrails	15
7.4 Where Automated Judges Fail	16
8. Limitations and Future Work	16
9. Conclusion	16
References	17

Abstract

Large Language Models (LLMs) are increasingly deployed as autonomous agents that reason, use tools, and act over multiple steps. Yet most hallucination benchmarks still evaluate only the final output, missing failures that originate in intermediate Thought–Action–Observation steps. This project presents **Trajel**, a framework and dataset for auditing trajectory-level hallucinations in multi-agent industrial workflows built on top of the AssetOpsBench benchmark (IBM Research). We introduce a five-type hallucination taxonomy — factual, referential, logical, procedural, and scope-based — and release a dataset of 225 expert-annotated agent traces labeled under this taxonomy. We benchmark three supervised detection paradigms (BERT, NLI, Longformer) and an LLM-as-a-Judge evaluator, and analyze five execution-quality signals as lightweight hallucination predictors. Our results show that the most common failure modes (procedural hallucinations, 38.5%) are invisible to output-only evaluation, that nearly half of hallucinated trajectories involve multiple types simultaneously (48.7%), and that automated detectors with high binary accuracy still misclassify the subtlest types. The clarity-and-justification execution signal achieves $AUC = 0.908$, outperforming all trained classifiers (best $AUC = 0.689$), suggesting that lightweight run-time monitors offer the most practical path to real-time hallucination detection in deployed agentic systems.

1. Introduction

The transition from static Large Language Models to autonomous agentic systems represents a fundamental frontier in artificial intelligence. In high-stakes industrial sectors such as data center monitoring and infrastructure maintenance, agents are no longer mere text generators — they are decision-making entities tasked with parsing multi-modal signals, following rigorous procedures, and coordinating across multi-agent frameworks. As these systems gain autonomy, they inherit a more complex and dangerous failure mode: **trajectory-level hallucination**. In an agentic context, a hallucination is not simply a factual confabulation in a single response — it is a structural deviation from evidence that propagates through a sequential, tool-mediated trajectory, often leading to cascading operational failures.

Despite the critical nature of these systems, the science of AI evaluation has remained largely tethered to static benchmarks. Current evaluation regimes for hallucination typically focus on “one-shot” tasks like summarization or question-answering, treating each instance as an isolated input–output pair. This paradigm fails to capture the temporal and interactive dynamics of an agent loop. In a multi-step workflow involving Thought, Action, and Observation cycles, a hallucination might surface as a procedural skip, a mis-referenced entity from a previous step, or an off-scope action that violates safety constraints.

This project addresses this gap by building on **AssetOpsBench** [1], IBM Research’s benchmark for AI-driven failure diagnosis of industrial assets. AssetOpsBench dispatches tasks across four FastMCP tool servers (IoT, FMSR, TSFM, and WO) using a Plan-Execute orchestrator to identify root causes, predict failure modes, and generate maintenance work orders. Each scenario produces a sequential Thought → Action → Observation trace — precisely the setting where trajectory-level evaluation matters most.

Our four contributions are:

1. **A Trajectory-Aware Hallucination Taxonomy.** We define five hallucination types — factual, referential, logical, procedural, and scope-based — as structural predicates over the Thought–Action–Observation execution trace, enabling diagnostic disentanglement of grounding failures from reasoning errors and control-flow violations.
2. **The Trajel Dataset.** 225 expert-annotated agent trajectories spanning 6 model configurations and 42 industrial AssetOps tasks, jointly labeled by LLM-as-a-Judge and blind human reviewers from Columbia University and IBM Research, with a human-identified hallucination rate of 68.3% and Cohen’s $\kappa = 0.456$.
3. **The Trajel ML Modeling Framework.** Benchmarking of three supervised detection paradigms — subtask-level BERT classification, trajectory-level NLI, and long-context Longformer — each motivated by the context requirements of specific hallucination types.
4. **Empirical Validation of Execution Signals.** The first systematic study of which execution-quality signals most reliably predict hallucination, finding that the clarity-and-justification signal achieves $AUC = 0.908$ as a univariate predictor, substantially

outperforming all trained classifiers.

2. Related Work

We review prior work on agent benchmarks, hallucination detection in tool-augmented settings, hallucination taxonomies, and trajectory-level evaluation, then position Trajel against existing benchmarks.

Agent benchmarks. ReAct [3] interleaves reasoning and action to expose intermediate traces, and AgentBench [4] demonstrates that LLM performance degrades sharply in long-horizon tasks. WebArena [10] provides realistic web environments, and HotpotQA [11] provides multi-hop reasoning chains — but large-scale human-labeled agent trajectory datasets remain scarce.

Hallucination in tool-augmented agents. ToolBH [6] and MIRAGE-Bench [7] show that hallucinations predominantly arise during intermediate reasoning and tool use rather than in final outputs. However, neither formalizes hallucination types as structural predicates over the execution trace, and both rely primarily on LLM-as-a-Judge evaluation, leaving supervised trajectory-level classifiers underexplored.

Hallucination taxonomies. Cognitive Mirage [8] categorizes factual, logical, and contextual errors; MIRAGE adds perception-versus-reasoning distinctions for multimodal settings; MIRAGE-Bench introduces an agentic taxonomy over instruction, history, and observation inconsistencies. None separate procedural violations (broken workflow ordering) from scope-based violations (correct content from the wrong agent) — a distinction essential in multi-agent industrial settings and central to our taxonomy.

Multi-agent failure analysis. Cemri et al. [13] introduce MAST, a 14-mode failure taxonomy validated on 1600+ annotated traces, covering system design, inter-agent misalignment, and task verification — but do not isolate hallucination as a distinct phenomenon. TRAJECT-Bench [14] evaluates tool-use correctness along agentic trajectories but does not address hallucination detection or multi-agent coordination.

Positioning Trajel. Table 1 situates Trajel against existing benchmarks. To our knowledge, Trajel is the first benchmark to combine industrial multi-agent trajectories with full trajectory-level evaluation, a structurally grounded five-type taxonomy, expert human annotations, and LLM-as-a-Judge baselines.

Table 1: Comparison of agentic evaluation benchmarks for hallucination analysis.

Benchmark	Industrial	Multi-Agent	Trajectory	Taxonomy	Human	LLM Judge
AgentBench [4]	No	Yes	No	No	No	No
WebArena [10]	No	Yes	No	No	Yes	No
MIRAGE-Bench [7]	No	No	Partial	Yes	No	Yes
ToolBH [6]	No	Yes	Partial	Yes	No	Yes
MAST [13]	No	Yes	Full	Partial	Yes	Yes
TRAJECT-Bench [14]	No	No	Full	No	No	No
AssetOpsBench [1]	Yes	Yes	Partial	No	Yes	Yes
Trajel (ours)	Yes	Yes	Full	Yes	Yes	Yes

3. Problem Formulation

3.1 Trajectory Structure

We model an agentic workflow as a compound AI system $\Phi = (M, C, T_{\text{tool}})$, where $M = \langle M_1, \dots, M_K \rangle$ is a set of LLM-driven agent modules, C is the orchestrator (Plan-and-Execute), and T_{tool} is the tool set. In AssetOpsBench, $K = 4$ with agents $\mathcal{A} = \{\text{IoT}, \text{FMSR}, \text{TSFM}, \text{WO}\}$ covering perception, state modeling, temporal forecasting, and execution.

Execution proceeds as a sequence of steps, each a Thought–Action–Observation triple produced by one agent: thought τ_t (reasoning), action α_t (tool invocation), and observation $\omega_t = T(\alpha_t)$.

Definition 1 (Trajectory). A step is $s_t = (a_t, \tau_t, \alpha_t, \omega_t)$ with $a_t \in \mathcal{A}$ and $t \in \{1, \dots, N\}$. A trajectory is the ordered trace $\mathcal{T} = (s_1, \dots, s_N)$. In AssetOpsBench, $\mathcal{T}$ is serialized as a single JSON array — a unified, causally ordered information stream in which every step has access to all prior evidence and may therefore reference, misreference, or fabricate upstream content.

3.2 Hallucination Taxonomy

A hallucination at step s_t is a deviation in τ_t or α_t from what is warranted by the evidence set $\mathcal{E}_t = \{\omega_1, \dots, \omega_{t-1}\}$, the task specification $\mathcal{K}$, and the agent’s role.

Definition 2 (Hallucination). Let $g_t \in \{\tau_t, \alpha_t\}$. A hallucination is the predicate:

$$h(g_t \mid \mathcal{E}_t, \mathcal{K}, a_t) = \mathbb{1}[g_t \not\models \mathcal{E}_t \vee g_t \not\models \mathcal{K} \vee g_t \not\models \text{role}(a_t)]$$

We refine this into five categories $\mathcal{H} = \{h^F, h^R, h^L, h^P, h^S\}$:

Table 2: Five-type hallucination taxonomy with formal definitions.

Type	Symbol	Definition
Factual	h^F	Asserts a claim contradicted by ground-truth data; detectable from a single step
Referential	h^R	References an entity or prior result absent from $\{s_1, \dots, s_{t-1}\}$
Logical	h^L	Reasoning in τ_t does not follow from its premises
Procedural	h^P	Skips, reorders, or fabricates a step required by $\mathcal{K}$
Scope	h^S	Agent a_t acts or claims outside its mandate $\text{role}(a_t)$

The types are mutually clarifying rather than disjoint: a single step may exhibit several simultaneously, so we model labels as multi-label vectors in $\{0, 1\}^{|\mathcal{H}|}$. They are also ordered by the context required for detection: h^F needs only the step and ground truth; h^R, h^L additionally require $\mathcal{E}_t$; h^P further requires $\mathcal{K}$; and h^S further requires $\text{role}(a_t)$. This ordering predicts the relative strengths of subtask-level versus trajectory-level detectors.

3.3 Detection Tasks

A **subtask-level detector** $f^{\text{sub}} : s_t \mapsto \{0, 1\}^{|\mathcal{H}|}$ produces per-step, per-category predictions. A **trajectory-level detector** $f^{\text{traj}} : \mathcal{T}_\Phi \rightarrow \{0, 1\}$ flags any trajectory containing a hallucination.

3.4 Research Questions

- **RQ1 (Prevalence):** What is the empirical distribution of hallucination types across $\mathcal{H}$, and are certain types concentrated in specific agents or trace positions?
 - **RQ2 (Localization):** Given a hallucinated trajectory, can we identify the originating step and distinguish hallucination from co-occurring execution errors?
 - **RQ3 (Detection Modeling):** How do subtask-level BERT, trajectory-level NLI, and long-context Longformer compare in detecting hallucinated trajectories?
 - **RQ4 (Predictive Signals):** Which execution-quality signals most reliably predict hallucination early enough to support real-time intervention?
-

4. Methodology

4.1 Evaluation Pipeline

Our pipeline operates on trajectories produced by AssetOpsBench and proceeds in three stages:

Stage 1 — Trajectory generation and labeling. We collect agent execution traces across AssetOpsBench task scenarios. Each trajectory is labeled at two granularities: (i) subtask-level, annotating individual steps s_t with hallucination type; and (ii) trajectory-level, assigning a binary hallucination label to the full trace $\mathcal{T}$. Labeling follows a two-phase protocol: an initial pass using LLM-as-a-Judge, followed by blind human review in which annotators assess trajectories without access to the LLM’s judgments.

Stage 2 — Prompt variation and stress-testing. We systematically modify evaluation prompts (altering instruction specificity, reordering sub-goals, varying procedural detail) to analyze how prompt variation influences hallucination frequency and type distribution, directly addressing RQ1.

Stage 3 — Detection and classification. Labeled trajectories are used to train and evaluate supervised detection models. We emphasize ROC-AUC as the primary evaluation metric for robustness under class imbalance.

4.2 Detection Modeling

The taxonomy’s context-ordering hypothesis motivates three complementary modeling paradigms:

Paradigm 1 — Subtask-level classification (BERT). A fine-tuned BERT classifier operates on individual steps s_t , receiving the concatenation of τ_t , α_t , and ω_t as input and predicting whether the step contains a hallucination. This paradigm captures local lexical cues and is most effective for factual hallucinations.

Paradigm 2 — Trajectory-level NLI. A natural language inference formulation treats hallucination detection as an entailment problem. For each step s_t , trajectory history $\{s_1, \dots, s_{t-1}\}$ serves

as the premise and the current step’s thought and action as the hypothesis. This paradigm targets referential and logical hallucinations requiring trace-wide consistency.

Paradigm 3 — Long-context modeling (Longformer). A Longformer classifier ingests the full serialized trajectory as a single input sequence, exploiting sparse attention to process traces exceeding standard context windows. This paradigm is designed for procedural and scope-based hallucinations requiring global context.

4.3 LLM-as-a-Judge

For scalable automated annotation, we employ Claude (claude-haiku-4-5-20251001) as an LLM judge. The judge receives the full trajectory — question, characteristic answer, agent reasoning trace, and final response — and returns a structured JSON prediction specifying hallucination presence, type(s), location, and rationale. The judge’s predictions are compared against blind human annotations to establish inter-rater agreement and identify systematic failure modes.

4.4 Signal Analysis

Beyond supervised classification, we investigate which execution-quality signals available during or immediately after agent execution are most predictive of hallucination (RQ4). We operationalize five signal families using AssetOpsBench evaluation dimensions:

- **Task Completion (TC):** Binary flag for whether the agent completed all required sub-tasks
- **Data Retrieval Accuracy (DRA):** Whether sensor data was correctly retrieved from IoT servers
- **Result Verification (RV):** Whether intermediate results were cross-validated before use
- **Agent Sequence Correctness (ASC):** Whether agents were invoked in the prescribed order
- **Clarity and Justification (CJ):** Whether reasoning steps were clearly grounded in observations

If sufficiently predictive, these signals could support lightweight real-time monitors integrated into the agent orchestration loop.

5. Trajel Dataset

5.1 Composition

Trajel comprises 225 annotated trajectories generated by the AssetOpsBench multi-agent framework. Each trajectory is a complete execution trace — a JSON-serialized sequence of Thought–Action–Observation steps across four domain agents (IoT, FMSR, TSFM, WO) — produced in response to one of 42 industrial operations questions covering sensor retrieval, anomaly detection, failure-mode identification, and work-order generation.

Table 3: Trajel dataset overview.

Statistic	Value
Total annotated trajectories	225
Unique model configurations	6
Unique task questions	42
Annotation institutions	2 (Columbia University, IBM Research)
Human-identified hallucination rate	68.3% (153 / 224)
LLM-judge-identified hallucination rate	79.1% (178 / 225)
Single-type hallucinations	79 (51.3%)
Multi-type hallucinations	75 (48.7%)

5.2 Annotation Protocol

Each trajectory is evaluated independently by two parties: an LLM-as-a-Judge and a human reviewer. Human reviewers annotate blind — without access to the LLM judge’s output — to prevent anchoring bias. For each trajectory, annotators record four fields: (1) hallucination presence, (2) hallucination type(s) from $\mathcal{H}$, (3) localization within the step and component, and (4) free-text rationale.

Review principles. Agent failure is not automatically classified as hallucination unless there is clear evidence of explicit information fabrication. Repetition alone does not indicate hallucination, and self-correcting behavior reflects adaptive reasoning. Model evaluations may be partially accurate, correctly detecting an issue while misidentifying its type or location.

Annotation procedure. Each trajectory is reviewed in sequence: (i) read task description; (ii) assess step count for overthinking indicators; (iii) verify task-agent alignment; (iv) evaluate thought-action-observation consistency; (v) record hallucination type and location; (vi) compare with LLM judge and note agreement/disagreement.

Human reviewers are drawn from Columbia University (148 reviews) and IBM Research (77 reviews), with comparable agreement rates across institutions (Columbia: 64.9%, IBM: 59.7%), confirming consistent annotation standards.

5.3 Annotation Quality

Table 3 reports the confusion matrix between LLM-judge and human annotations ($n = 224$).

Table 4: LLM-as-a-Judge vs. human annotation confusion matrix. The judge achieves 92.2% recall but only 79.7% precision, reflecting a conservative bias that favors over-flagging over missed detections. Cohen’s $\kappa = 0.456$ (moderate agreement).

	Human: No Hall.	Human: Hall.
LLM Judge: No Hall.	35	12
LLM Judge: Hall.	36	141

Per-type Cohen’s κ (Table 4) confirms the context-ordering hypothesis: types requiring only local evidence (scope $\kappa = 0.656$, procedural $\kappa = 0.613$, factual $\kappa = 0.595$) show moderate-to-substantial agreement, while types requiring cross-step reasoning (logical $\kappa = 0.211$, referential $\kappa = 0.176$) show only slight agreement.

Table 5: Per-type Cohen’s κ between LLM-as-a-Judge and human annotators.

Type	Both	Agent Only	Human Only	Neither	Cohen’s κ
Scope (h^S)	33	9	16	167	0.656
Procedural (h^P)	78	26	17	104	0.613
Factual (h^F)	50	24	15	136	0.595
Logical (h^L)	4	6	17	198	0.211
Referential (h^R)	3	7	14	201	0.176

5.4 Type Distribution

Table 6: Distribution of hallucination types (human evaluation). Procedural hallucinations are the most prevalent category at 38.5%.

Type	Count	%
Procedural (h^P)	95	38.5%
Factual (h^F)	65	26.3%
Scope (h^S)	49	19.8%
Logical (h^L)	21	8.5%
Referential (h^R)	17	6.9%
Total occurrences	247	

Procedural hallucinations dominate at 38.5%. This is significant: procedural failures — skipping

required diagnostic steps, fabricating workflow completion, acting on nonexistent tool outputs — are invisible to methods that check only factual accuracy. Notably, 48.7% of hallucinated trajectories exhibit multiple types simultaneously, validating the multi-label formulation and confirming that single-label classification would mischaracterize nearly half of all real failures.

5.5 Cross-Model Analysis

Table 7: Per-model hallucination rates and type profiles. Rates range from 52.4% to 81.0%, confirming substantial variation across architectures on identical tasks.

Model	N	Hall. Rate	h^F	h^R	h^L	h^P	h^S
Model 6	42	52.4%	7	0	2	17	2
Model 19	42	64.3%	9	6	2	16	18
Model 7	41	68.3%	13	2	6	17	6
Model 12	42	71.4%	13	1	4	20	6
Model 16	15	80.0%	5	1	1	7	4
Model 17	42	81.0%	18	7	6	18	13

Hallucination rates vary substantially across models on the same 42-question task suite (52.4%–81.0%). Model 19 exhibits a disproportionate share of scope-based hallucinations (18 of 27 hallucinated trajectories), while Model 6 shows almost none (2 of 22). These patterns would be invisible under a binary hallucination label, reinforcing the diagnostic value of the five-type taxonomy.

5.6 Localization Analysis

Hallucinations are most frequently localized to **Actions** (70) and **Responses** (58) — the externally visible components — rather than Thoughts (40) or Observations (11). This suggests the primary failure mode is the translation of reasoning into tool invocations and output claims, with direct implications for guardrail design. Hallucinations also peak at Task 3 (63 occurrences), consistent with increased context accumulation mid-trajectory.

6. Experiments and Results

6.1 Experimental Setup

All analyses are conducted on the 224 trajectories with complete human annotations. The LLM-as-a-Judge serves as the primary automated baseline; its predictions are compared against human labels across all five hallucination types. We report precision, recall, and F1 per type, and ROC-AUC for supervised models.

6.2 Per-Type Detection Quality (RQ1)

Table 8: LLM-as-a-Judge per-type detection performance against human annotations.

Type	TP	FP	FN	Precision	Recall	F1
Procedural (h^P)	78	26	17	0.750	0.821	0.784
Factual (h^F)	50	24	15	0.676	0.769	0.719
Scope (h^S)	32	9	16	0.780	0.667	0.719
Logical (h^L)	4	6	17	0.400	0.190	0.258
Referential (h^R)	3	7	14	0.300	0.176	0.222

The LLM judge achieves $F1 = 0.784$ on procedural and 0.719 on factual hallucinations, but only 0.258 and 0.222 on logical and referential types. This disparity is consistent with the context-ordering hypothesis: procedural and factual hallucinations have more overt surface cues, while logical and referential hallucinations require deeper cross-step reasoning. The judge over-flags procedural hallucinations (26 FP vs. 17 FN) but primarily under-detects referential and logical types.

6.3 Binary vs. Taxonomy-Aware Evaluation (RQ2)

At the binary level, the LLM judge agrees with human annotators on 176 of 224 trajectories (78.6%). However, among the 141 trajectories where both identified a hallucination, exact agreement on type occurs in only 82 cases (58.2%). The mean Jaccard similarity between type sets is 0.746 .

For logical hallucinations, the judge’s miss rate is 79% (agreed on 4 of 19 human-identified instances); for referential types, 77%. Under binary evaluation, all 141 trajectories would be counted as correctly detected, obscuring systematic failure on the subtlest types. This validates the multi-label taxonomy as a necessary evaluation instrument: binary accuracy of 78.6% masks a type-level miss rate exceeding 75% for the most nuanced hallucination categories.

6.4 Supervised Detection Models (RQ3)

Table 9: Supervised detection model comparison. All three fine-tuned models improve over pre-trained baselines, confirming learnable signal in Trajel. The NLI model achieves the highest AUC (0.689), consistent with the prediction that trajectory-level context benefits models operating over the full trace.

Model	Precision	Recall	F1	ROC-AUC
Majority-class baseline	0.683	1.000	0.812	0.500
LLM-as-a-Judge (zero-shot)	0.797	0.922	0.855	—
BERT (subtask-level)	0.548	0.639	0.590	0.613
NLI (trajectory-level)	0.500	0.643	0.563	0.689
Longformer (long-context)	0.471	0.615	0.533	0.599

All fine-tuned models improve over their pre-trained baselines, confirming that Trajel contains learnable signal. The NLI model achieves the highest ROC-AUC (0.689), indicating that trajectory-level context improves ranking quality even when thresholded metrics remain modest. However, all supervised models fall short of the zero-shot LLM judge on F1 (0.855), indicating that 225 trajectories are insufficient to match the general reasoning capability of large prompted models and motivating dataset scaling.

6.5 Signal Analysis (RQ4)

Table 10: Execution-quality signals as univariate predictors of hallucination ($n = 213$). CJ is the strongest single predictor (AUC = 0.908, $r = -0.833$).

Signal	Hall. Rate (present)	Hall. Rate (absent)	Pearson r	AUC
Task completion (TC)	2.3%	90.0%	−0.786	0.853
Data retrieval accuracy (DRA)	43.0%	89.6%	−0.502	0.771
Result verification (RV)	6.4%	91.0%	−0.784	0.863

Signal	Hall. Rate (present)	Hall. Rate (absent)	Pearson r	AUC
Agent sequence correct (ASC)	43.7%	86.6%	−0.453	0.738
Clarity & justifica- tion (CJ)	9.1%	94.3%	−0.833	0.908

All five signals are strongly negatively correlated with hallucination. The clarity-and-justification signal is the strongest univariate predictor (AUC = 0.908): trajectories lacking grounded justification hallucinate at 94.3%, versus only 9.1% for high-clarity reasoning. Critically, CJ alone outperforms the best supervised classifier (NLI AUC = 0.689). When both CJ and RV signals are absent simultaneously, the hallucination rate reaches 97.1%, suggesting an effective kill-switch condition for agent orchestrators.

7. Discussion

7.1 Why Procedural Hallucinations Dominate

Procedural hallucinations (38.5%) are the dominant failure mode because AssetOpsBench tasks require strict diagnostic workflows — IoT retrieval before TSFM inference, FMSR analysis before WO generation. Agents that skip, reorder, or fabricate steps can produce plausible-sounding outputs that pass factual checks while the underlying workflow is broken. This is precisely the failure mode invisible to output-only evaluation.

7.2 The Multi-Label Problem

With 48.7% of hallucinated trajectories exhibiting multiple types simultaneously, single-label classification would mischaracterize nearly half of all real failures. The most common co-occurring pairs — procedural-factual ($n = 27$) and procedural-scope ($n = 26$) — indicate that control-flow violations frequently cascade into other failure modes, validating the multi-label formulation.

7.3 Execution Signals as Practical Guardrails

The CJ signal (AUC = 0.908) substantially outperforms all trained classifiers (best AUC = 0.689). This shifts the practical recommendation: rather than deploying heavyweight classifiers, agent orchestrators should integrate lightweight execution-quality monitors that flag trajectories when clarity and justification signals fail. The 97.1% hallucination rate when both CJ and RV are absent provides a near-certain kill-switch condition.

7.4 Where Automated Judges Fail

The LLM judge’s per-type F1 drops to 0.258 (logical) and 0.222 (referential), exposing a gap between aggregate binary accuracy (78.6%) and reliable type-level diagnosis. This divergence reflects genuine annotator ambiguity — even human reviewers show $\kappa < 0.25$ for these types — suggesting that improving detection of referential and logical hallucinations will require richer annotation protocols alongside more powerful modeling.

8. Limitations and Future Work

Dataset scope. Trajel comprises 225 trajectories from a single industrial domain. While this exercises all five hallucination types, the distribution may differ in healthcare, finance, or open-ended web tasks. The six model configurations share a common orchestration framework; behavior under alternative orchestrators remains untested.

Annotation quality. Inter-annotator agreement is moderate overall (Cohen’s $\kappa = 0.456$) and only slight for referential and logical types ($\kappa \leq 0.211$). A larger and more diverse annotator pool, along with richer annotation protocols requiring annotators to trace referential claims back to specific prior steps, would strengthen ground-truth reliability.

Model limitations. The supervised detection models (BERT, NLI, Longformer) are benchmarked on a relatively small labeled set. Generalization to unseen domains and architectures requires further investigation. The gap between supervised AUC (0.689) and LLM-judge F1 (0.855) motivates hybrid architectures combining LLM-derived features with discriminative classifiers.

Future directions. Scaling to additional domains, investigating token-level uncertainty and semantic entropy as additional predictive signals, extending to multi-model ensembles where inter-agent disagreement can be measured directly, and developing richer annotation protocols for referential and logical types are the primary next steps.

9. Conclusion

We introduced **Trajel**, a trajectory-aware framework and dataset for hallucination detection in multi-agent industrial workflows. Our five-type taxonomy — factual, referential, logical, procedural, and scope-based — formalizes hallucination as a structural predicate over the Thought–Action–Observation execution trace, enabling diagnostic analysis that binary labels cannot provide.

Empirical evaluation on 225 expert-annotated trajectories across six model configurations reveals three principal findings. First, procedural hallucinations are the dominant failure mode (38.5%), are invisible to output-only evaluation, and validate the need for trajectory-level benchmarks. Second, nearly half of hallucinated trajectories (48.7%) exhibit multiple types simultaneously, confirming

that single-label formulations mischaracterize a large share of real failures. Third, automated detectors with high binary accuracy (LLM judge $F1 = 0.855$) systematically misclassify the subtlest types (per-type κ as low as 0.176 for referential hallucinations), exposing a gap between aggregate performance and reliable type-level diagnosis.

Signal analysis reveals that execution-quality flags available during the agent loop are strongly predictive of hallucination: the clarity-and-justification signal alone achieves $AUC = 0.908$, substantially outperforming supervised classifiers (best $AUC = 0.689$). This shifts the practical recommendation from post-hoc classification toward lightweight runtime monitors integrated into the agent orchestration loop.

We hope that Trajel will serve as a foundation for future work on transparent, auditable, and safer multi-agent deployment in high-stakes industrial settings. The dataset, annotation protocol, LLM-as-a-Judge prompt, and detection code are publicly available at github.com/aishani-rachakonda/genai-using-llms-final-project.

References

- [1] Patel, D., Lin, S., Rayfield, J., Zhou, N., Vaculin, R., Martinez, N., O'donncha, F., Kalagnanam, J. (2025). AssetOpsBench: Benchmarking AI Agents for Task Automation in Industrial Asset Operations and Maintenance. *arXiv preprint arXiv:2506.03828*.
- [2] Agrawal, L.A., Tan, S., Soylu, D., et al. (2026). GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. *ICLR 2026 (Oral)*. *arXiv:2507.19457*.
- [3] Yao, S., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*.
- [4] Liu, X., et al. (2023). AgentBench: Evaluating LLMs as Agents. *arXiv preprint arXiv:2308.03688*.
- [5] Darwish, A.M., et al. (2025). Mitigating LLM Hallucinations Using a Multi-Agent Framework. *Information*, 16(7):517.
- [6] Wu, et al. (2024). ToolBH: A Multi-level Hallucination Diagnostic Benchmark for Tool-Augmented LLMs. *EMNLP 2024*.
- [7] Zhang, W., et al. (2025). MIRAGE-Bench: LLM Agent is Hallucinating and Where to Find Them. *arXiv preprint arXiv:2507.21017*.
- [8] Ye, H., et al. (2023). Cognitive Mirage: A Review of Hallucinations in Large Language Models. *arXiv preprint arXiv:2309.06794*.
- [9] Dong, B., et al. (2025). MIRAGE: Assessing Hallucination in Multimodal Reasoning Chains of MLLMs. *arXiv preprint arXiv:2505.24238*.

- [10] Deng, X., et al. (2024). WebArena: A Realistic Web Environment for Building Autonomous Agents. *arXiv preprint arXiv:2307.13854*.
- [11] Yang, Z., et al. (2018). HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. *EMNLP 2018*.
- [12] Lin, S., et al. (2022). TruthfulQA: Measuring How Models Mimic Human Falsehoods. *ACL 2022*.
- [13] Cemri, M., et al. (2025). Why Do Multi-Agent LLM Systems Fail? *NeurIPS 2025, Datasets and Benchmarks Track*.
- [14] He, P., et al. (2026). TRAJECT-Bench: A Trajectory-Aware Benchmark for Evaluating Agentic Tool Use. *ICLR 2026*.